

[WIP] 🍴 Luncher

An enterprise agent for planning team lunch meetings

Demo application for use in  [TEMPLATE] Agentic Application SDLC Workshop - MAKE A COPY

Product

Problem statement: Scheduling is hard!

Our organization needs to plan regular team meetings where we discuss strategically important issues. In order to maximize attendance, we'll provide a free lunch. How can we find a day/time that works for the most possible attendees, and ensure that we serve food that works for everyone, while crafting an agenda that makes good use of everyone's time?

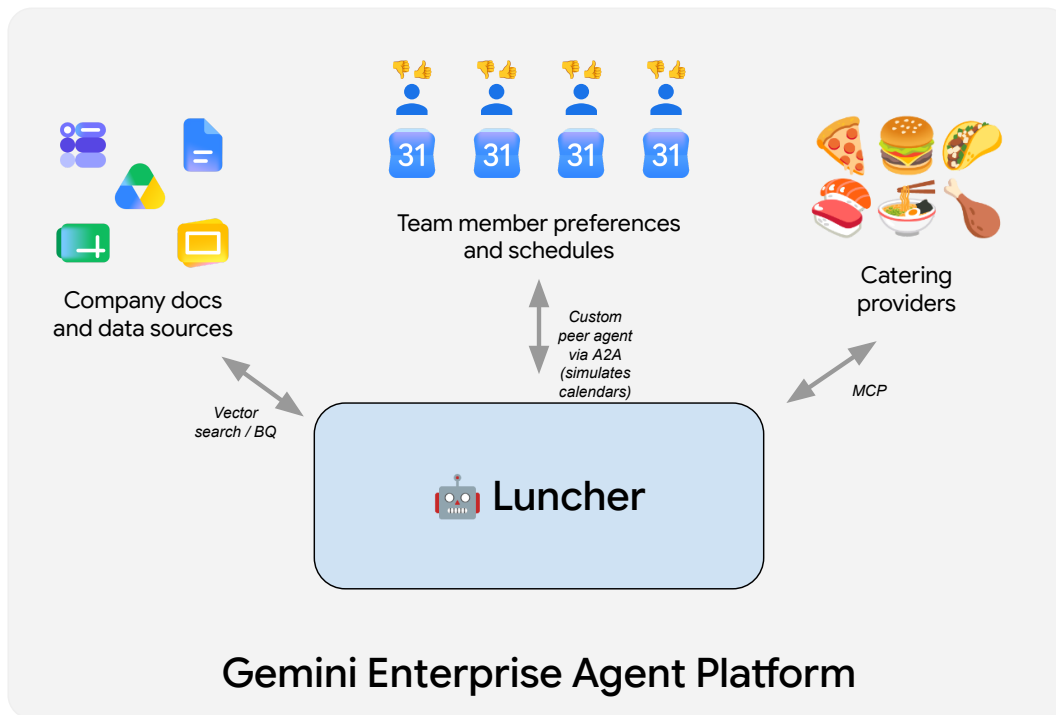
User Personas

 Meeting planner (one per meeting) an administrative professional who is in charge of coordinating the meeting	 Attendee (many per meeting) team members who will attend
--	---

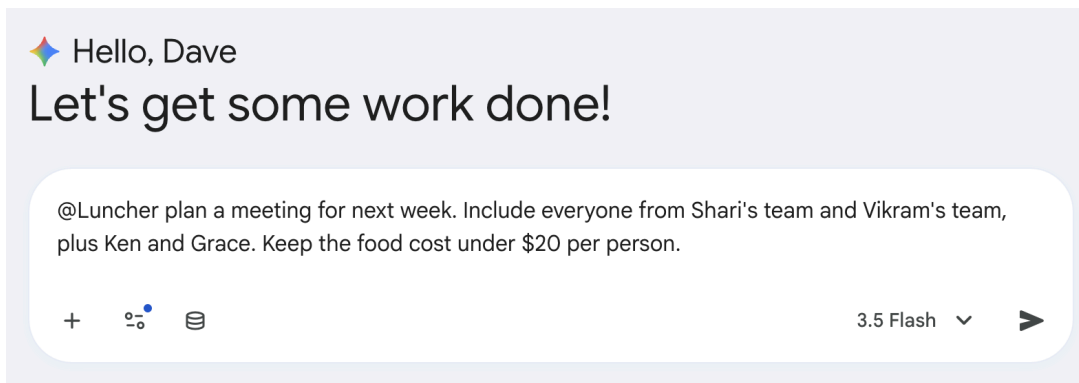
CUJs

- **Meeting planner**
 - **Plan a meeting:** input parameters (date range, desired attendees, general topic area)
- **Meeting planner; attendee**
 - **Specify preferences:** describe preferences for food, meeting time

Markecture



Core Interface (via Gemini Enterprise App)



Technical Implementation

Components

- **Orchestration agent**
 - The root agent, built with ADK, which receives the Meeting Planner's request and coordinates the response
 - hosted in Agent Runtime
 - Exposed via Gemini Enterprise App
 - [tentative] Uses A2UI for rich user experience
- **Knowledge connectors [out of the box]**
 - Pre-built connectors in Gemini Enterprise, including Workspace [and others?], to provide context on organizational goals and activities
- **Team preference agent**
 - Custom agent which provides schedule availability and individual preferences (including food-related) for team members

- Hosted in Agent Runtime
- Leverages GEAP-managed memories
- Uses Model Armor to protect team member privacy (e.g. food allergies may be considered HIPAA protected information)
- Communicates to the Orchestration agent via A2A
- **Catering MCP**
 - Custom service providing menus and food ordering API for catering
 - Menus and order status etc. can be mocked and stored as in-memory data for simplicity
 - Hosted on Cloud Run
 - Registered in GEAP MCP Service registry
- **[stretch] SDLC autonomous agent(s)**
 - One or more agents (default: one agent for code review) hosted on GEAP and using the AntigraVity SDK to interact with repo code
 - Triggered via webhook
 - See: <https://github.com/davidstanke/microservices-demo/blob/main/.github/workflows/vertex-ai-agent.yml> for an example (of issue triage, not code review, but the idea is the same)

GCP services/products used

- AntigraVity 2.0 and CLI
- ADK & Agents CLI
- Gemini Enterprise
- GEAP: Agent Registry
- GEAP: MCP Registry
- GEAP: Agent Runtime
 - Including Trace, O11y
- GEAP: Memory Bank
- Model Armor
- Cloud Run
- [stretch] AntigraVity SDK

Out of scope [reason]

- Agent Gateway [not widely released; missing key functionality]
- CodeMender [not widely released]
- Cloud Build / Cloud Deploy [not high priority for customer adoption; sticking to GHA keeps it simple]
- GKE [we don't need another runtime]

Open questions

- *Crowdsourced memory?* Can we make it so that each employee provides their own (possibly PII) food (or other) preferences?
 - [dave] IDK if an agent invoked by **person_a** can access memories stored on behalf of **person_b** ... ideally, **person_a** can invoke the agent and get results informed by **person_b**'s preferences, but can't actually see **person_b**'s preferences.

- If not, we can still demonstrate memory by just making `person_a` manage all of it: “remember that `person_b` doesn’t like cilantro.”
- *CRUD?* As described, this agent will *create* an event, and remember peoples’ preferences. But what about *modifying* events? Should we add that functionality? If so, should there be a datastore for the orchestrator, or should it just treat the planner’s calendar as its data store [in which case, we may want to create some sort of tag in the event so it can be identified later, e.g. `#createdbyluncher`]
 - [dave] I think this should be at most a stretch goal. Let’s focus on creating these events and not worry about what happens after. I just parked this idea because it came to mind.